

LeetCode 480: Sliding Window Median

Comprehensive Notes & Optimal Solutions

1. Complexity Overview

Finding the median of a sliding window of size k efficiently requires maintaining a sorted structure as the window moves.

- **Brute Force:** Sorting every window individually.
Time Complexity: $O(n \times k \log k)$
Space Complexity: $O(k)$
- **Optimal Approaches:** Trees/Heaps where insertion and deletion take $O(\log k)$.
Time Complexity: $O(n \log k)$
Space Complexity: $O(k)$

2. Approach 1: Two Multisets

Core Idea: We need a data structure that can maintain sorted order and allow duplicate values. A C++ `multiset` is perfect for this.

- Maintain 2 multisets: one for low scores (l) and one for high scores (r).
- **Balancing Rule:** Ensure no side contains significantly more elements than the other. In the case of an odd window size (k), the right multiset (r) is allowed to store exactly 1 more element than the left.
- **Rebalancing:** If the left multiset has more elements, take its largest element (last element) and add it to the beginning of the right multiset. Multiset handles the internal sorting automatically.
- **Insertion:** If the incoming element is greater than the first element of the right multiset, insert it to the right; otherwise, insert it to the left. Finally, balance.
- **Deletion:** Remove the outgoing element from whichever multiset currently holds it, then rebalance.

Implementation: Two Multisets

```

class Solution {
public:
    multiset<int> l, r;
    int n;

    void balance() { // Ensure neither half has extra elements improperly
        while(l.size() > r.size()) {
            r.insert(*l.rbegin());
            l.erase(prev(l.end())); // Cannot use l.erase(l.rbegin()) as erase expects normal iterator
        }

        while(r.size() > l.size() + 1) { // Right side can hold one extra element for odd k
            l.insert(*r.begin());
            r.erase(r.begin());
        }
    }

    vector<double> medianSlidingWindow(vector<int>& nums, int k) {
        vector<double> ans;
        n = nums.size();

        for(int i = 0; i < n; i++) {
            if(r.empty() || nums[i] > *r.begin()) {
                r.insert(nums[i]);
            } else {
                l.insert(nums[i]);
            }

            balance();

            if(i >= k) { // Remove outgoing elements
                if(l.find(nums[i-k]) != l.end()) {
                    l.erase(l.find(nums[i-k]));
                } else {
                    r.erase(r.find(nums[i-k]));
                }
                balance();
            }

            if(i >= k - 1) { // Valid window
                if(k % 2 == 1) {
                    ans.push_back(*r.begin());
                } else {
                    ans.push_back(((double)*r.begin() + (double)*l.rbegin()) / 2.0);
                }
            }
        }
    }
}

```

```
        return ans;
    }
};
```

3. Approach 2: Two Heaps + Lazy Deletion

Core Idea: Similar to the multiset method, maintain two heaps. One max-heap (`small`) for the smaller half and one min-heap (`large`) for the larger half.

The Challenge: Let the current window be `[5, 8, 10]`. The max-heap might look like `[10, 8, 5]`. If 5 becomes outgoing, a standard heap cannot easily remove it because it only allows access to the top element.

The Solution (Lazy Deletion): Maintain an unordered map (`lazy`) for deletions.

We do `lazy[5]++`. This implies the heap temporarily stores expired elements. Whenever an expired element eventually reaches the top of the heap, we perform a deferred (lazy) pop.

Note: We maintain `smallSize` and `largeSize` exclusively to count the VALID elements, disregarding the lazily deleted "zombie" nodes.

Implementation: Two Heaps + Lazy Deletion

```

class Solution {
public:
    int n;
    priority_queue<int> small; // maxheap for smaller half
    priority_queue<int, vector<int>, greater<int>> large; // minheap for larger half

    unordered_map<int, int> lazy; // tracks elements waiting for deletion
    int scnt = 0; // count of valid elements in small
    int lcnt = 0; // count of valid elements in large

    void prunesmall() { // execute lazy deletion on maxheap top
        while(!small.empty()) {
            int x = small.top();
            if(lazy[x] == 0) {
                break;
            }
            lazy[x]--;
            small.pop();
        }
    }

    void prunelarge() { // execute lazy deletion on minheap top
        while(!large.empty()) {
            int x = large.top();
            if(lazy[x] == 0) {
                break;
            }
            lazy[x]--;
            large.pop();
        }
    }

    void balance() {
        if(scnt > lcnt + 1) {
            large.push(small.top());
            small.pop();
            lcnt++;
            scnt--;
            prunesmall(); // Invalid elements might surface on small after popping
        }
        else if(lcnt > scnt) {
            small.push(large.top());
            large.pop();
            lcnt--;
            scnt++;
            prunelarge(); // Invalid elements might surface on large after popping
        }
    }
}

```

```

}

void insertn(int num) {
    if(small.empty() || num <= small.top()) {
        small.push(num);
        scnt++;
    }
    else {
        large.push(num);
        lcnt++;
    }
    balance();
}

void deletion(int num) {
    lazy[num]++; // mark for future deletion

    if(num <= small.top()) {
        scnt--;
        if(num == small.top()) {
            prunesmall(); // immediate deletion possible
        }
    }
    else {
        lcnt--;
        if(num == large.top()) {
            prunelarge(); // immediate deletion possible
        }
    }
    balance();
}

double medianlo(int k) {
    if(k % 2 == 1) {
        return small.top();
    }
    else {
        return ((double)small.top() + (double)large.top()) / 2.0;
    }
}

vector<double> medianSlidingWindow(vector<int>& nums, int k) {
    n = nums.size();
    vector<double> ans;

    for(int i = 0; i < n; i++) {
        insertn(nums[i]);
    }
}

```

```
        if(i - k >= 0) {
            deletion(nums[i-k]);
        }

        if(i >= k - 1) {
            ans.push_back(medianlo(k));
        }
    }
    return ans;
}

};
```